

머신러닝

데이터 전처리

Chapter 5



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 데이터 전처리의 기초

02 데이터 전처리의 전략

03 데이터 전처리의 실습

01

데이터 처리의 기초



데이터 전처리의 개념

- 데이터 전처리(Data Preprocessing)

: 머신러닝 모델에 데이터를 입력하기 전에 데이터를 정제하고 가공하는 과정

- 넘파이와 판다스 같은 데이터 처리 도구

- 맷플롯립과 시본 같은 데이터 시각화 도구를 활용하여 실제 데이터를 정리

- 머신러닝 기초 수식

$$y = f(X)$$

- 데이터 X 를 모델 함수 $f(X)$ 에 입력하면 결과 y 가 출력됨

데이터 X 는 훈련 데이터(train data)와 테스트 데이터(test data)가 모두 같은 구조를 갖는 피쳐(feature)이어야 함



데이터 품질 문제

- 머신러닝 모델의 성능을 높이기 위해서는 데이터의 품질 문제를 먼저 해결

1) 데이터 분포의 지나친 차이

- 데이터가 연속형 값인데 최댓값과 최솟값 차이가 다른 피쳐보다 더 많이 나는 경우
- 학습에 영향을 줄 수 있기 때문에 데이터의 스케일을 맞춰줌=>스케일링
 - 데이터의 최댓값과 최솟값을 0에서 1 사이 값으로 바꾸거나 표준 정규분포 형태로 나타내는 등

2) 범주형 데이터

- 범주형 데이터의 명목형 및 순서형 데이터는 일반적으로 숫자로 표현되지 않음
- 컴퓨터가 이해할 수 있는 숫자 형태의 정보로 변형



데이터 품질 문제

- 머신러닝 모델의 성능을 높이기 위해서는 데이터의 품질 문제를 먼저 해결

3) 결측치 (missing data) : 실제로 존재하지만 데이터베이스 등에 기록되지 않는 데이터

- 해당 데이터를 그대로 사용할 수 없기 때문에 결측치 처리 전략을 세워 데이터를 채워 넣음

4) 이상치 (outlier) : 극단적으로 크거나 작은 값

- 단순히 데이터 분포의 차이와는 다름
- 데이터 오기입이나 특이 현상 때문에 나타남
- 결측치 해결방식과 유사

02

데이터 전처리의 전략



1) 데이터 분포 스케일링

- 스케일링(scaling) : 데이터 간 범위를 맞춤
 - 몸무게와 키를 하나의 모델에 넣으면 데이터의 범위가 훨씬 넓어져 키가 몸무게에 비해 모델에 과다하게 영향을 줌
- 열(x_1)과 열(x_2)의 범위가 다를 때 하나의 변수 범위로 통일시켜 처리



```
import pandas as pd
import numpy as np

df = pd.DataFrame(
    {'A': [14.00, 90.20, 90.95, 96.27, 91.21],
     'B': [103.02, 107.26, 110.35, 114.23, 114.68],
     'C': ['big', 'small', 'big', 'small', 'small']})
```

df



	A	B	C
0	14.00	103.02	big
1	90.20	107.26	small
2	90.95	110.35	big
3	96.27	114.23	small
4	91.21	114.68	small



1) 데이터 분포 스케일링: 최솟값-최댓값 정규화

- 최솟값-최댓값 정규화(min-max normalization) :

최솟값과 최댓값을 기준으로 0에서 1, 또는 0에서 지정 값까지로 값의 크기를 변화시킴

$$z_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- x 는 처리하고자 하는 열
- x_i 는 해당 열의 하나의 값
- $\max(x)$ 는 해당 열의 최댓값, $\min(x)$ 는 해당 열의 최솟값

```
(df["A"] - df["A"].min() ) / (df["A"].max() - df["A"].min())
```

```
0    0.000000
1    0.926219
2    0.935335
3    1.000000
4    0.938495
```

```
Name: A, dtype: float64
```

▶ 1) 데이터 분포 스케일링: z-스코어 정규화

- z-스코어 정규화(z-score normalization) : 기존 값을 표준 정규 분포값(평균 0, 표준편차 1)으로 변환하여 처리

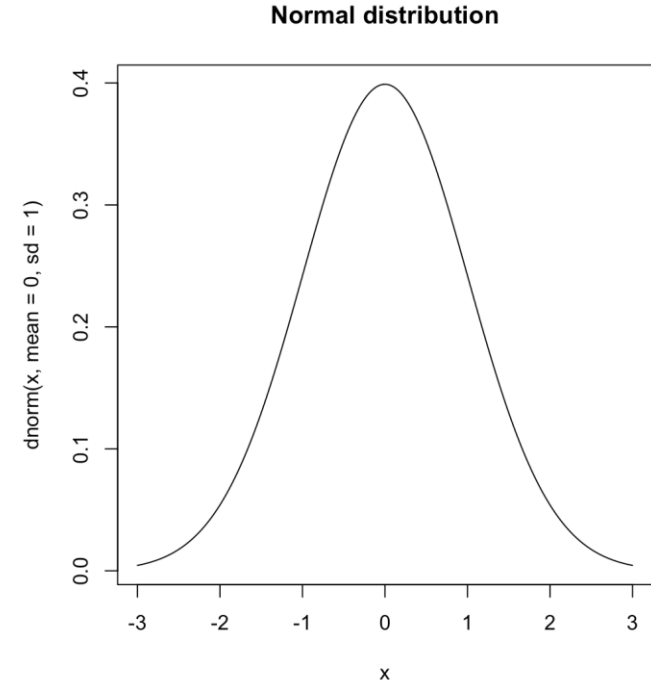
$$z = \frac{x_i - \mu}{\sigma}$$

- μ 는 x 열의 평균값이고 σ 는 표준편차

```
(df["B"] - df["B"].mean() ) / (df["B"].std())
```

```
0    -1.405250
1    -0.540230
2     0.090174
3     0.881749
4     0.973556
```

```
Name: B, dtype: float64
```





2) 범주형 데이터 처리: 원핫인코딩

- 원핫인코딩(one-hot encoding): 범주형 데이터의 개수만큼 가변수(dummy variable)를 생성하여 해당 범주에 속하면 1, 그렇지 않으면 0으로 표현
 - color라는 변수에 {Green, Blue, Yellow} 3개의 값이 있을 때
 - 3개의 가변수를 만들고 각 색상에 인덱스를 지정
 - Green의 인덱스는 0, Blue의 인덱스 1, Yellow의 인덱스는 2로 지정
 - 해당 값이면 1, 아니면 0을 입력

{Green} → [1, 0, 0]

{Blue} → [0, 1, 0]

{Yellow} → [0, 0, 1]

```
edges = pd.DataFrame({'source': [0, 1, 2],  
                      'target': [2, 2, 3],  
                      'weight': [3, 4, 5],  
                      'color': ['red', 'blue', 'blue']})  
edges
```



	source	target	weight	color
0	0	2	3	red
1	1	2	4	blue
2	2	3	5	blue



2) 범주형 데이터 처리: 원핫인코딩

- get_dummies 함수 사용
 - 범주형 데이터(categorical data)를 숫자형 데이터로 변환할 때 사용
 - 숫자형이 아닌 데이터 타입에 적용

```
print(edges.dtypes)
pd.get_dummies(edges)
```

```
source      int64
target      int64
weight      int64
color       object
dtype: object
```

[8]:

	source	target	weight	color_blue	color_red
0	0	2	3	False	True
1	1	2	4	True	False
2	2	3	5	True	False



2) 범주형 데이터 처리: 원핫인코딩

- 필요에 따라 정수형을 객체로 변경해서 처리
 - weight는 숫자로 되어 있지만 기수형 데이터
 - 데이터를 M, L, XL로 변경하여 원핫인코딩을 적용

```
weight_dict = {3:"M", 4:"L", 5:"XL"}  
edges["weight_sign"] = edges["weight"].map(weight_dict)  
weight_sign = pd.get_dummies(edges["weight_sign"])  
weight_sign
```

```
pd.concat([edges, weight_sign], axis=1)
```



	L	M	XL
0	False	True	False
1	True	False	False
2	False	False	True

	source	target	weight	color	weight_sign	L	M	XL
0	0	2	3	red	M	False	True	False
1	1	2	4	blue	L	True	False	False
2	2	3	5	blue	XL	False	False	True

- 데이터를 원핫인코딩 형태로 변경한 후 필요에 따라 병합이나 연결로 두 가지의 데이터를 합침



2) 범주형 데이터 처리: 바인딩

- 바인딩(binding) : 연속형 데이터를 범주형 데이터로 변환

```
raw_data = {'regiment': ['Nighthawks', 'Nighthawks', 'Nighthawks', 'Nighthawks',  
                        'Dragoons', 'Dragoons', 'Dragoons', 'Dragoons', 'Scouts', 'Scouts', 'Scouts', 'Scouts'],  
            'company': ['1st', '1st', '2nd', '2nd', '1st', '1st', '2nd', '2nd', '1st', '1st', '2nd', '2nd'],  
            'name': ['Miller', 'Jacobson', 'Ali', 'Milner', 'Cooze', 'Jacon',  
                    'Ryaner', 'Sone', 'Sloan', 'Piger', 'Riani', 'Ali'],  
            'preTestScore': [4, 24, 31, 2, 3, 4, 24, 31, 2, 3, 2, 3],  
            'postTestScore': [25, 94, 57, 62, 70, 25, 94, 57, 62, 70, 62, 70]}
```

```
df = pd.DataFrame(raw_data)
```

```
df
```

	regiment	company	name	preTestScore	postTestScore
0	Nighthawks	1st	Miller	4	25
1	Nighthawks	1st	Jacobson	24	94
2	Nighthawks	2nd	Ali	31	57
3	Nighthawks	2nd	Milner	2	62
4	Dragoons	1st	Cooze	3	70
5	Dragoons	1st	Jacon	4	25
6	Dragoons	2nd	Ryaner	24	94
7	Dragoons	2nd	Sone	31	57
8	Scouts	1st	Sloan	2	62
9	Scouts	1st	Piger	3	70
10	Scouts	2nd	Riani	2	62
11	Scouts	2nd	Ali	3	70



2) 범주형 데이터 처리: 바인딩

- postTestScore에 대한 학점을 측정하는 코드를 작성
 - 데이터 범위를 구분 : 0~25, 25~50, 50~75, 75~100으로 구분
 - 함수 cut 사용
 - bins 리스트에 구간의 시작 값 끝 값을 넣고 구간의 이름을 리스트로 나열
bins의 원소는 5개이고 group_names는 4개
 - cut 함수로 나눌 시리즈 객체와 구간, 구간의 이름을 넣어주면 해당 값을 바인딩하여 표시해줌

```
bins = [0, 25, 50, 75, 100] # bins 정의(0-25, 25-50, 50-75, 75-100)
group_names = ['Low', 'Okay', 'Good', 'Great']
categories = pd.cut(df['postTestScore'], bins, labels=group_names)
categories
#열 추가
#df['categories']=categories
```



```
0      Low
1     Great
2      Good
3      Good
4      Good
5      Low
6     Great
7      Good
8      Good
9      Good
10     Good
11     Good
Name: postTestScore, dtype: category
Categories (4, object): ['Low' < 'Okay' < 'Good' < 'Great']
```



3) 결측치 처리

■ 드롭과 채우기

- 데이터를 삭제(드롭)하거나 데이터를 채움
- 데이터가 없으면 해당 행이나 열을 삭제
- 평균값, 최빈값, 중간값 등으로 데이터를 채움

```
import pandas as pd
import numpy as np

raw_data = {'first_name': ['Jason', np.nan, 'Tina', 'Jake', 'Amy'],
            'last_name': ['Miller', np.nan, 'Ali', 'Milner', 'Cooze'],
            'age': [42, np.nan, 36, 24, 73],
            'gender': ['m', np.nan, 'f', 'm', 'f'],
            'preTestScore': [4, np.nan, np.nan, 2, 3],
            'postTestScore': [25, np.nan, np.nan, 62, 70]}

df = pd.DataFrame(raw_data)
df
```

	first_name	last_name	age	gender	preTestScore	postTestScore
0	Jason	Miller	42.0	m	4.0	25.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Tina	Ali	36.0	f	NaN	NaN
3	Jake	Milner	24.0	m	2.0	62.0
4	Amy	Cooze	73.0	f	3.0	70.0



3) 결측치 처리

- 결측치를 확인할 때 isnull 함수 사용
 - NaN 값이 존재할 경우 True, 그렇지 않을 경우 False 출력
 - sum 함수로 True인 경우 모두 더하고 전체 데이터 개수로 나누어 열별 데이터 결측치 비율을 구함

```
df.isnull()
```

	first_name	last_name	age	gender	preTestScore	postTestScore
0	False	False	False	False	False	False
1	True	True	True	True	True	True
2	False	False	False	False	True	True
3	False	False	False	False	False	False
4	False	False	False	False	False	False

```
df.isnull().sum() / len(df)
```

```
first_name    0.2
last_name     0.2
age           0.2
gender        0.2
preTestScore  0.4
postTestScore 0.4
dtype: float64
```



3) 결측치 처리: 드롭

- 드롭(drop) : 결측치가 나온 열이나 행을 삭제
- dropna 사용하여 NaN이 있는 모든 데이터의 행을 없앴 (기본값: axis=0)

```
df.dropna()
```



	first_name	last_name	age	gender	preTestScore	postTestScore
0	Jason	Miller	42.0	m	4.0	25.0
3	Jake	Milner	24.0	m	2.0	62.0
4	Amy	Cooze	73.0	f	3.0	70.0

- 매개변수 how로 조건에 따라 결측치를 지움
 - 'all': 행에 있는 모든 값이 NaN일 때 해당 행을 삭제
 - 'any': 하나의 NaN만 있어도 삭제(기본값)

```
df_cleaned = df.dropna(how='all') # 행 삭제
df_cleaned
```

	first_name	last_name	age	gender	preTestScore	postTestScore
0	Jason	Miller	42.0	m	4.0	25.0
2	Tina	Ali	36.0	f	NaN	NaN
3	Jake	Milner	24.0	m	2.0	62.0
4	Amy	Cooze	73.0	f	3.0	70.0

```
df['location'] = np.nan
df.dropna(axis=1, how='all') # 열 삭제
```

	first_name	last_name	age	gender	preTestScore	postTestScore
0	Jason	Miller	42.0	m	4.0	25.0
1	NaN	NaN	NaN	NaN	NaN	NaN
2	Tina	Ali	36.0	f	NaN	NaN
3	Jake	Milner	24.0	m	2.0	62.0
4	Amy	Cooze	73.0	f	3.0	70.0



3) 결측치 처리: 드롭

- 매개변수 thresh 데이터의 개수를 기준으로 삭제
 - thresh=1:데이터가 한 개라도 존재하는 행은 남김
 - thresh=4:데이터가 4개 이상 있어야 남김

```
df.dropna(thresh=1) #행 기준
```

	first_name	last_name	age	gender	preTestScore	postTestScore	location
0	Jason	Miller	42.0	m	4.0	25.0	NaN
2	Tina	Ali	36.0	f	NaN	NaN	NaN
3	Jake	Milner	24.0	m	2.0	62.0	NaN
4	Amy	Cooze	73.0	f	3.0	70.0	NaN

```
df.dropna(axis=1, thresh=4) #열 기준
```

	first_name	last_name	age	gender
0	Jason	Miller	42.0	m
1	NaN	NaN	NaN	NaN
2	Tina	Ali	36.0	f
3	Jake	Milner	24.0	m
4	Amy	Cooze	73.0	f



3) 결측치 처리: 채우기

- 채우기(fill) : 비어있는 값을 채움
- 평균, 최빈값(빈도가 가장 높은 값) 등 데이터의 분포를 고려해서 채움
- 함수 fillna 사용
 - 지정한 값으로 채움
 - inplace: 변경된 값을 리턴시키는 것이 아니고 해당 변수 자체의 값을 변경

```
df.fillna(0)
```

	first_name	last_name	age	gender	preTestScore	postTestScore	location
0	Jason	Miller	42.0	m	4.0	25.0	0.0
1	0	0	0.0	0	0.0	0.0	0.0
2	Tina	Ali	36.0	f	0.0	0.0	0.0
3	Jake	Milner	24.0	m	2.0	62.0	0.0
4	Amy	Cooze	73.0	f	3.0	70.0	0.0

```
df.fillna({"preTestScore": 0}, inplace=True) #권장  
df["postTestScore"].fillna(df["postTestScore"].mean(), inplace=True)  
df
```

	first_name	last_name	age	gender	preTestScore	postTestScore	location
0	Jason	Miller	42.0	m	4.0	25.000000	NaN
1	NaN	NaN	NaN	NaN	0.0	52.333333	NaN
2	Tina	Ali	36.0	f	0.0	52.333333	NaN
3	Jake	Milner	24.0	m	2.0	62.000000	NaN
4	Amy	Cooze	73.0	f	3.0	70.000000	NaN



3) 결측치 처리: 채우기

- 열별 분포를 고려하여 채울 수 있음
 - groupby 함수로 각 인덱스의 성별에 따라 빈칸을 채움
 - fillna 함수 안에 transform을 사용하여 인덱스를 기반으로 채울 수 있음

```
temp = df.groupby("gender")["postTestScore"].transform("mean")
df.fillna({"postTestScore": temp}, inplace=True)
print(temp)
print(df)
```

```
0    43.500000
1         NaN
2    61.166667
3    43.500000
4    61.166667
```

Name: postTestScore, dtype: float64

	first_name	last_name	age	gender	preTestScore	postTestScore	location
0	Jason	Miller	42.0	m	4.0	25.000000	NaN
1	NaN	NaN	NaN	NaN	0.0	52.333333	NaN
2	Tina	Ali	36.0	f	0.0	52.333333	NaN
3	Jake	Milner	24.0	m	2.0	62.000000	NaN
4	Amy	Cooze	73.0	f	3.0	70.000000	NaN

03

데이터 전처리의 실습



머신러닝 프로세스와 데이터 전처리

- 데이터를 확보한 후 데이터를 정제 및 전처리

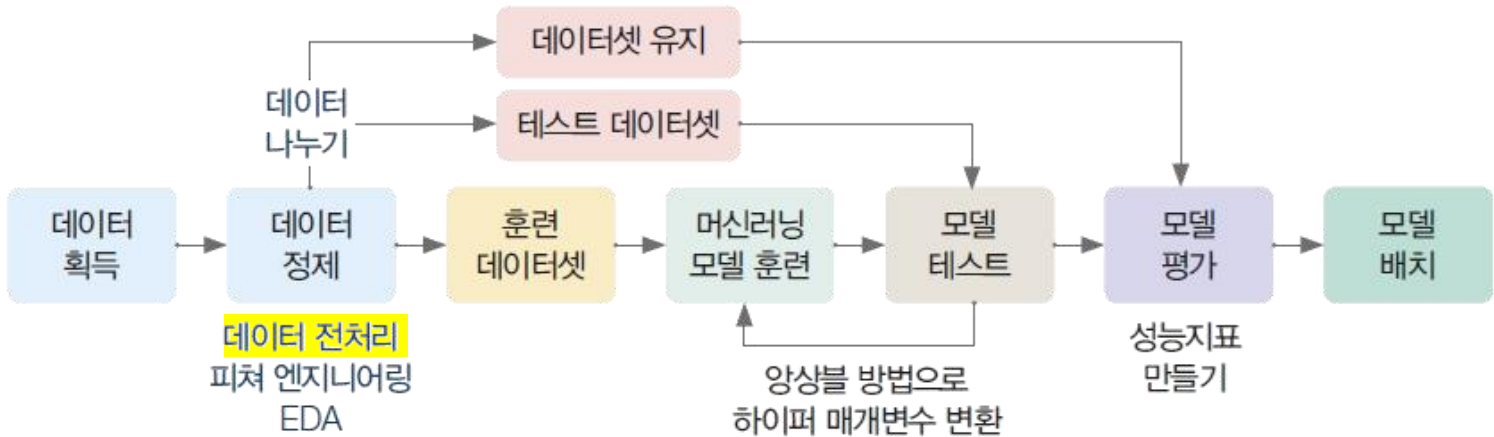


그림 6-3 머신러닝 프로세스

- 데이터 정제/데이터 전처리 단계는 실제로 가장 많은 시간이 들어가는 작업



그림 6-4 머신러닝 시스템 개발의 업무 비중 © D Scully

- 머신러닝 모델을 만드는 'ML Code' 작업은 가장 작은 부분을 차지하고 데이터 수집이나 피처 추출 (Feature Extraction), 데이터 검증(Data Verification)이 훨씬 더 많은 부분을 차지



머신러닝을 위한 오픈 데이터 활용

- 머신러닝 모델을 학습하기 위해서는 다양한 데이터셋이 필요하며 이를 위해 공개된 오픈 데이터(Open Data)를 활용하는 경우가 많음
 - 누구나 접근 가능하여 데이터 수집 비용이 들지 않음
 - 다양한 분야의 대규모 데이터셋을 쉽게 확보 가능
 - 머신러닝 연구 및 실습에서 표준적으로 사용되는 데이터 제공
 - 동일한 데이터를 사용하여 연구 결과를 비교 및 재현 가능

사이트	특징	주소
Kaggle	가장 유명한 머신러닝 데이터 플랫폼 대회 + 데이터셋 제공	https://www.kaggle.com
Dacon	한국 데이터 분석 경진대회 플랫폼	https://dacon.io
UCI Machine Learning Repository	머신러닝 연구용 대표 데이터셋 저장소	https://archive.ics.uci.edu
Google Dataset Search	구글이 제공하는 데이터셋 검색 엔진	https://datasetsearch.google.com
Open Data Portal (data.go.kr)	한국 정부 공공데이터 포털	https://www.data.go.kr
AWS Open Data	대규모 공개 데이터셋 제공	https://registry.opendata.aws



타이타닉 데이터 획득

- test.csv : 예측되는 탑승객들의 데이터가 있는 파일
- train.csv : 모델을 학습시키기 위한 데이터가 있는 파일

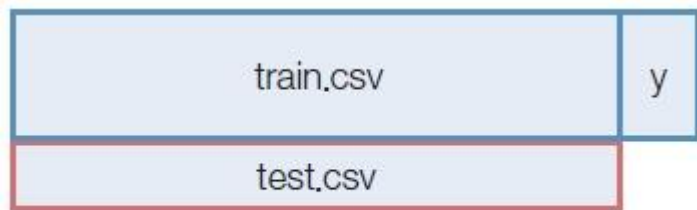


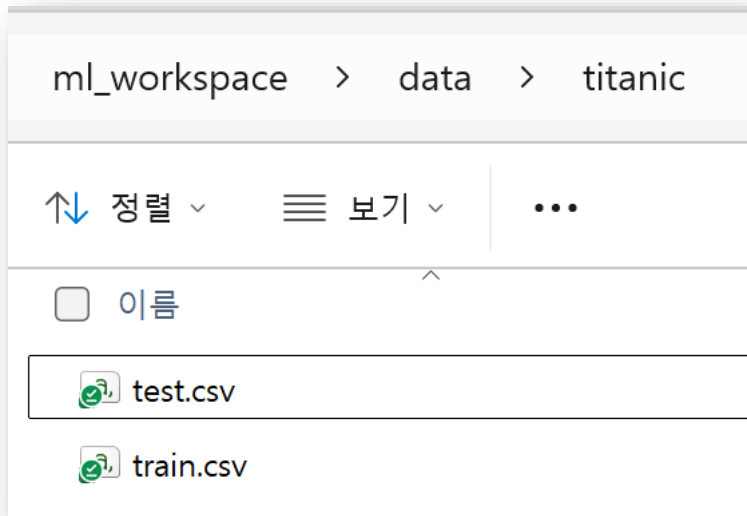
그림 6-9 train.csv와 test.csv의 관계

- 'train.csv' 파일과 달리 'test.csv' 파일에는 y 값, 즉 탑승객의 생존 유무에 대한 열이 없음



데이터 확인하기

- 다운로드한 파일의 압축을 풀고 아래와 같이 gender_submission.csv 파일은 삭제



```
import pandas as pd
import os
import matplotlib.pyplot as plt
import numpy as np

DATA_DIR = './data/titanic'

data_files = sorted([os.path.join(DATA_DIR, filename)
                     for filename in os.listdir(DATA_DIR)], reverse=True)
data_files
```

```
['./data/titanic\\train.csv', './data/titanic\\test.csv']
```



데이터 확인하기

- 2개의 파일로 나뉘어진 데이터를 하나의 데이터 프레임으로 만들기

```
# (1) 데이터프레임을 각 파일에서 읽어온 후 df_list에 추가
```

```
df_list = []
```

```
for filename in data_files:
```

```
    df_list.append(pd.read_csv(filename))
```

```
# (2) 두 개의 데이터프레임을 하나로 통합
```

```
df = pd.concat(df_list)
```

```
# (3) 인덱스 초기화
```

```
df = df.reset_index(drop=True)
```

```
# (4) 결과 출력
```

```
df.head(5)
```

	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0.00	3	Braund, Mr. Owen Harris	male	22.00	1	0	A/5 21171	7.25	NaN	S
1	2	1.00	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.00	1	0	PC 17599	71.28	C85	C
2	3	1.00	3	Heikkinen, Miss. Laina	female	26.00	0	0	STON/O2. 3101282	7.92	NaN	S
3	4	1.00	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.00	1	0	113803	53.10	C123	S
4	5	0.00	3	Allen, Mr. William Henry	male	35.00	0	0	373450	8.05	NaN	S



데이터 확인하기

- 보통 다른 피쳐들을 기반으로 Survived를 예측함

변수명	의미	값 종류
PassengerId	승객 ID	정수
Survived	생존 여부	0 = No, 1 = Yes
Pclass	티켓 클래스	1 = 1st, 2 = 2nd, 3 = 3rd
Name	승객 이름	문자열
Gender	성별	male, female
Age	나이	숫자
SibSp	타이타닉 밖의 형제자매/배우자의 수	숫자
Parch	타이타닉 밖의 부모/자식의 수	숫자
Ticket	티켓 번호	문자열
Fare	티켓 가격	숫자
Cabin	객실 번호	문자열
Embarked	승선 항구	C = Cherbourg Q = Queenstown S = Southampton



데이터 노트

- 데이터 노트 : 분석해야 하는 데이터에 대한 여러 가지 아이디어를 정리하는 노트
 - 각 데이터의 현재 데이터 타입 올바르게 정의
 - 숫자로 표시되어 있지만 범주형 데이터로 변형이 필요한 경우 등

변수명	의미	데이터 타입	아이디어
Survived	생존 여부	범주형 (이진)	Y 데이터
Pclass	티켓 클래스	범주형 (순서형)	좌석 등급이 생존율에 영향을 줄까?
Gender	성별	범주형	성별에 따른 생존율 차이가 있을까?
Age	나이	연속형 (float)	나이가 생존 여부에 영향을 줄까?
SibSp	형제자매 배우자 수	이산형 (int)	가족 수가 생존율에 영향을 줄까?
Parch	부모/자식 수	이산형 (int)	
Ticket	티켓 번호	식별자 / 문자열	분석 변수로 사용하기 어려움
Fare	티켓 가격	연속형 (float)	티켓 가격과 Pclass는 관련이 있을까?
Cabin	객실 번호	범주형 / 문자열	
Embarked	승선 항구	범주형	승선 항구와 생존율의 관계

df.head(1).T

	0	1
PassengerId	1	2
Survived	0.00	1.00
Pclass	3	1
Name	Braund, Mr. Owen Harris	Cumings, Mrs. John Bradley (Florence Briggs Th...
Gender	male	female
Age	22.00	38.00
SibSp	1	1
Parch	0	0
Ticket	A/5 21171	PC 17599
Fare	7.25	71.28
Cabin	NaN	C85
Embarked	S	C



결측치 확인하기

- 열별로 결측치 비율을 확인하여 전략을 세움

```
# (1) 데이터를 소수점 두 번째 자리까지 출력(생략가능)
pd.options.display.float_format = '{:.2f}'.format

# (2) 결측치 값의 합을 데이터의 개수로 나눠 비율로 출력
df.isnull().sum() / len(df) * 100
```

```
PassengerId    0.00
Survived       31.93
Pclass         0.00
Name           0.00
Gender         0.00
Age           20.09
SibSp          0.00
Parch          0.00
Ticket         0.00
Fare           0.08
Cabin         77.46
Embarked       0.15
dtype: float64
```

- Cabin 변수는 약 77%의 결측치가 존재하므로 데이터 활용이 어려움
- Age 변수는 약 20%의 결측치가 존재하여 대체 과정이 필요함
- Embarked 변수는 결측치 비율이 0.15%로 매우 적어 간단한 보완 가능
- 그 외 변수들은 결측치가 존재하지 않음

각 열의 특성에 맞는 전처리 전략이 필요함



결측치 전략: 삭제

- 결측치 비율이 높아 Cabin 열을 데이터셋에서 제거함

```
df.drop('Cabin', axis=1, inplace=True)
df.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Gender', 'Age', 'SibSp',  
      'Parch', 'Ticket', 'Fare', 'Embarked'],  
      dtype='object')
```

- Survived 열은 예측해야 하는 중요한 변수이므로 결측치가 존재하는 행은 데이터셋에서 제거

```
df.dropna(subset=["Survived"], inplace=True)
df.tail()
```

	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Embarked
886	887	0.00	2	Montvila, Rev. Juozas	male	27.00	0	0	211536	13.00	S
887	888	1.00	1	Graham, Miss. Margaret Edith	female	19.00	0	0	112053	30.00	S
888	889	0.00	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	S
889	890	1.00	1	Behr, Mr. Karl Howell	male	26.00	0	0	111369	30.00	C
890	891	0.00	3	Dooley, Mr. Patrick	male	32.00	0	0	370376	7.75	Q



결측치 전략: 평균값 사용

- 범주형 변수 기준으로 그룹을 나누고 결측치를 제외한 평균값으로 결측치를 대체함

```
df[df["Age"].notnull()].groupby(["Gender"])["Age"].mean()
```

```
Gender
female    27.92
male      30.73
Name: Age, dtype: float64
```

```
df[df["Age"].notnull()].groupby(["Pclass"])["Age"].mean()
```

```
Pclass
1    38.23
2    29.88
3    25.14
Name: Age, dtype: float64
```

나이(Age)의 결측치를 대체할 때 성별(Gender) 기준 평균보다 좌석 등급(Pclass) 기준 평균을 사용하는 것이 보다 정확한 추정

```
df.fillna({"Age":df.groupby("Pclass")["Age"].transform("mean")}, inplace=True)
df.isnull().sum() / len(df) * 100
```

- `transform("mean")`: 각 행에 자신이 속한 그룹의 평균을 다시 붙여줌



```
PassengerId    0.00
Survived       0.00
Pclass         0.00
Name           0.00
Gender         0.00
Age            0.00
SibSp          0.00
Parch          0.00
Ticket         0.00
Fare           0.00
Embarked       0.22
dtype: float64
```




결측치 전략: 최빈값 사용

- Embarked가 결측치인 행인 극소수이므로 조회하기

```
df[df['Embarked'].isnull()]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
61	62	1.00	1	Icard, Miss. Amelie	female	38.00	0	0	113572	80.00	NaN
829	830	1.00	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.00	0	0	113572	80.00	NaN

- Embarked의 최빈값 찾고 그 값으로 결측치 채우기

```
count = df.groupby("Embarked")["Embarked"].count()
count
```



```
Embarked
C      168
Q       77
S     644
Name: Embarked, dtype: int64
```

```
#비추천 코드
df.iloc[[61,829],10] = "S"
df.iloc[[61,829],10]
```



```
61      S
829      S
```



원핫인코딩

- 데이터 형태에 따라 처리 방법 결정
- `df.info()` 함수 : 열별로 데이터 타입을 확인

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 891 entries, 0 to 890
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	float64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Gender	891 non-null	object
5	Age	891 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Embarked	891 non-null	object

```
dtypes: float64(3), int64(4), object(4)
```

```
memory usage: 83.5+ KB
```



원핫인코딩

```
ctg_columns = ["Pclass", "Gender", "Embarked"]
```

```
for c in ctg_columns:
```

```
    onehot = pd.get_dummies(df[c], prefix=c)#해당 열 원핫인코딩
```

```
    #index로 병합하여 다시 df에 저장 df = pd.concat([df, onehot], axis=1)
```

```
    df = pd.merge(df,onehot, how="inner", left_index=True, right_index=True)
```

```
df.head(2).T
```

	0	1
PassengerId	1	2
Survived	0.00	1.00
Pclass	3	1
Name	Braund, Mr. Owen Harris	Cumings, Mrs. John Bradley (Florence Briggs Th...
Gender	male	female
Age	22.00	38.00
SibSp	1	1
Parch	0	0
Ticket	A/5 21171	PC 17599
Fare	7.25	71.28
Embarked	S	C
Pclass_1	False	True
Pclass_2	False	False
Pclass_3	True	False
Gender_female	False	True
Gender_male	True	False
Embarked_C	False	True
Embarked_Q	False	False
Embarked_S	True	False



데이터 시각화

- 각 변수와 생존 여부(Survived)의 관계를 그래프로 나타내어 생존에 영향을 주는지 시각적으로 확인

```
import seaborn as sns
import matplotlib.pyplot as plt

#ctg_columns = ["Pclass", "Gender", "Embarked"]

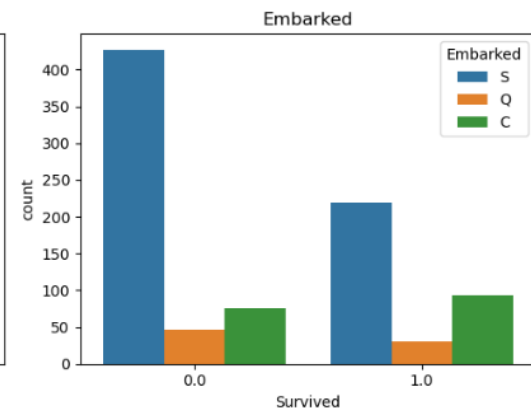
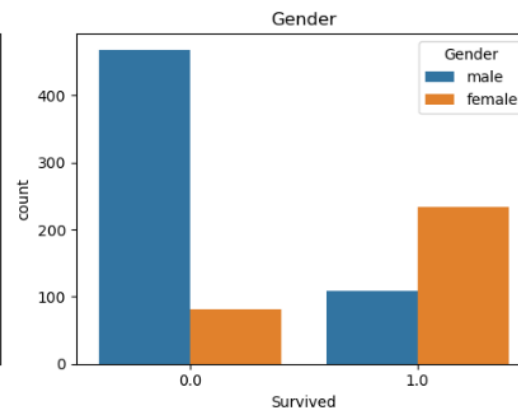
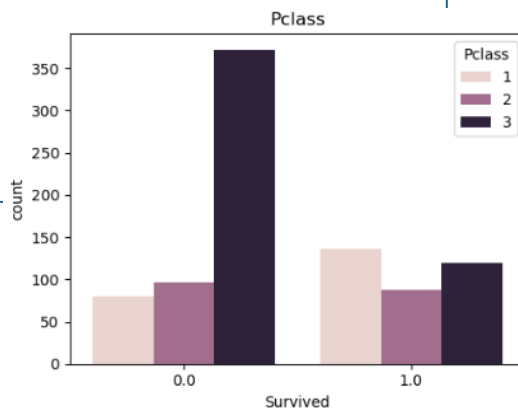
#크기 지정
plt.figure(figsize=(15,4))

for i in range(len(ctg_columns)):
    plt.subplot(1,3,i+1)#위치

    #그래프 그리기
    sns.countplot(x="Survived", hue=ctg_columns[i], data=df)

    #그래프 제목
    plt.title(ctg_columns[i])

plt.show()
```





SUMMARY

- 데이터 전처리(Data Preprocessing)
 - : 머신러닝 모델에 데이터를 입력하기 전에 데이터를 정제하고 가공하는 과정임
 - 스케일링: 변수 간 범위 차이를 줄여 모델 학습에 미치는 영향을 조정하는 과정
 - Min-Max 정규화
 - z-score 정규화
 - 범주형 데이터 처리 : 원핫 인코딩(one-hot encoding)
 - 결측치 처리: 삭제(drop)하거나 평균값, 최빈값 등 데이터 분포를 고려하여 채우는 방식
- 실제 데이터 분석에서는 공개된 오픈 데이터(Open Data)를 활용하여 데이터를 수집
- 타이타닉 데이터 실습
 - 결측치 처리
 - 범주형 데이터 변환
 - 데이터 시각화: 생존 여부와 변수 간 관계 분석



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝